

# Embedded Firmware with Zephyr RTOS

## Embedded Firmware with Zephyr RTOS

### A Practical Guide from LED to Custom PCB

**Author:** Jovin

**Target hardware:** Nordic nRF52840 DK

**Zephyr version:** 4.x

**SDK:** Zephyr SDK 1.0.1

---

**Embedded Firmware with Zephyr RTOS** is a hands-on, 24-day guide for engineers who want to build real firmware on the nRF52840 — not just blink an LED, but ship production-ready code.

Each day introduces one concept, one working example, and one buildable project. You start with the Zephyr build system and Device Tree basics, then progress through threads, GPIO interrupts, USB communication, CBOR encoding, and BLE advertising. By the end, you will have written a custom I2C accelerometer driver, streamed live sensor data to a PC over USB, defined your own PCB board target, and put the processor into sub- $\mu$ A sleep — the same techniques used in commercial wearables and IoT products.

**What's inside:** - 24 structured lessons with working Zephyr source code for every day - Custom driver development for sensors not supported by upstream Zephyr - USB CDC-ACM, CBOR encoding, and a Python decoder tool for live data visualization - BLE peripheral advertising, SPI flash, ADC, PWM, NVS storage, and power

management - A complete custom PCB board definition — ready to adapt to your own hardware - Setup guide covering Zephyr 4.x, West, and SDK 1.0.1 on Linux

**Who this is for:** Embedded engineers with C experience who are new to Zephyr, developers moving from bare-metal or Arduino to an RTOS, and hardware engineers writing firmware for their own nRF52840-based designs.

**Hardware required:** Nordic nRF52840 DK and a USB cable.

---

# Day 1 — Introduction to Zephyr RTOS

## Goal

Learn what Zephyr is, why it exists, and how it differs from other embedded RTOS options.

## What you will learn

- What Zephyr RTOS is and its core use cases
- How Zephyr supports multiple architectures and boards
- The difference between Zephyr application code and Zephyr kernel/SDK
- The role of West, CMake, Kconfig, and Device Tree in Zephyr

## Overview

Zephyr is a small, scalable real-time operating system for embedded devices. It is designed for constrained hardware and supports a wide range of microcontrollers, sensors, networking stacks, and boards.

## Key concepts

- **Kernel:** task scheduling, threads, synchronization, and timers
- **Subsystems:** networking, file systems, Bluetooth, sensor frameworks

- **Build system:** Zephyr uses CMake and West to manage projects and dependencies
- **Configuration:** Kconfig controls compile-time features and board-specific settings
- **Device Tree:** describes hardware peripherals and pin mappings

## Why this matters

Understanding these concepts helps you read Zephyr examples, choose the right board and SDK, and avoid common mistakes when starting a new project.

## Practice tasks

1. Inspect the Zephyr repository or samples to identify core directories: `zephyr/kernel`, `zephyr/subsys`, `zephyr/samples`.
2. Compare Zephyr to a bare-metal project: note what Zephyr provides out of the box.
3. Make a list of one board and one feature you want to explore next.

## Example folder

View the complete source code on GitHub: [src/2026-05-16 Introduction to Zephyr RTOS](#)

## Next topic

**Tomorrow, we will explore Zephyr project structure and the build system in depth.**

# Day 2 — Zephyr Project Structure and Build System

## Goal

Understand how a Zephyr application is organized and how the build system compiles it.

## What you will learn

- The role of `CMakeLists.txt` and `prj.conf`
- How West manages Zephyr projects and sample repositories
- The layout of a typical Zephyr application
- How to build and run a sample application

## Overview

A Zephyr application consists of source files, build metadata, and configuration files. The Zephyr build system uses CMake to generate build targets and West as a project manager.

## Typical files

- `CMakeLists.txt`: defines the application build, sources, and dependencies
- `prj.conf`: selects Zephyr features with Kconfig symbols
- `src/main.c`: application entry point
- `boards/` or board-specific overlays: optional hardware configuration

## Build flow

1. `west init` / `west update` to fetch Zephyr and modules
2. `west build -b <board> <app-dir>` to configure and compile
3. `west flash` or `west debug` to deploy and run

## Why this matters

Knowing the build structure helps you create valid Zephyr apps and troubleshoot build failures.

## Practice tasks

1. Create a minimal Zephyr app directory with `CMakeLists.txt`, `prj.conf`, and `src/main.c`.
2. Build a sample for a supported board such as `native_posix` or your target board.
3. Open the generated `build/zephyr/zephyr.elf` or build log to inspect compile steps.

## Example folder

View the complete source code on GitHub: [src/2026-05-17\\_Zephyr\\_Project\\_Structure](#)

## Next topic

**Tomorrow, we will cover Device Tree basics and how Zephyr maps hardware in software.**

# Day 3 — Device Tree Basics in Zephyr

## Goal

Learn the fundamentals of Zephyr Device Tree and how hardware is described for your board.

## What you will learn

- What a Device Tree is and why Zephyr uses it
- The structure of .dts and overlay files
- How Zephyr uses devicetree.h macros to access hardware
- Common node properties and bindings

## Overview

Device Tree is a declarative hardware description language. In Zephyr, it describes peripherals, pin assignments, and board-specific configuration without hard-coding them in C.

## Key concepts

- **Node:** a hardware block such as i2c0, gpio0, or uart1
- **Property:** settings like reg, status, label, pinctrl-0
- **Overlay:** custom modifications on top of the base board tree
- **Bindings:** JSON files that define allowed properties for a node type

## Common macros

- `DT_NODELABEL(...)`
- `DT_ANY_INST_ON_BUS(...)`
- `DEVICE_DT_GET(...)`
- `DT_PROP(...)`

## Why this matters

Device Tree allows portable hardware code, so the same application can run on different boards with only config changes.

## Practice tasks

1. Open a board's `.dts` file and locate an I2C or GPIO node.
2. Create an `app.overlay` that enables a peripheral and changes its status to okay.
3. Write a tiny `main.c` that fetches the device with `DEVICE_DT_GET`.

## Example folder

View the complete source code on GitHub: [src/2026-05-18 Device Tree Basics](#)

## Next topic

**Tomorrow, we will study Kconfig and `prj.conf` configuration in Zephyr.**

# Day 4 — Kconfig and `prj.conf` Configuration

## Goal

Understand how Zephyr features are enabled and configured using Kconfig and `prj.conf`.

## What you will learn

- The purpose of Kconfig in Zephyr
- How to enable features with `CONFIG_*` symbols
- How to search for configuration options
- How `prj.conf` affects the build

## Overview

Zephyr uses Kconfig to manage thousands of compile-time options. `prj.conf` is the application-specific file where you select the features your project needs.

## Examples

- `CONFIG_LOG=y` enables the logging subsystem
- `CONFIG_UART_CONSOLE=y` enables console output over UART
- `CONFIG_I2C=y` enables the I2C API
- `CONFIG_MAIN_STACK_SIZE=1024` sets thread stack size



## Why this matters

Correct configuration helps reduce firmware size, avoid conflicts, and enable the right drivers and services.

## Practice tasks

1. Open `zephyr/Kconfig` or use `west build -t menuconfig` if available.
2. Add logging and console output to `prj.conf`.
3. Try enabling and disabling a feature to see how it affects compile output.

## Example folder

View the complete source code on GitHub: [src/2026-05-19\\_Kconfig\\_prj\\_conf](#)

## Next topic

**Tomorrow, we will write a basic Zephyr application with main loop and device initialization.**

# Day 5 — Writing a Basic Zephyr Application

## Goal

Build a simple Zephyr application from scratch and understand the entry point and runtime model.

## What you will learn

- How `main()` works in Zephyr

- Using `printk()` and `LOG_*` for output
- Basic thread usage and delays
- Initializing devices at runtime

## Overview

A Zephyr app typically starts in `main()`, which runs as a thread under the Zephyr kernel. From there, you can initialize peripherals, configure timers, and start application logic.

## Typical code

- `void main(void)` is the app entry point
- `printk()` writes to the system console
- `k_sleep(K_SECONDS(x))` pauses the current thread
- `device_get_binding()` or `DEVICE_DT_GET` obtains peripheral handles

## Why this matters

This is the core of application development: writing code that interacts with the kernel and hardware.

## Practice tasks

1. Create `src/main.c` with a blinking LED or periodic console message.
2. Add a thread using `K_THREAD_DEFINE` or `k_thread_create`.
3. Use `k_sleep()` and `printk()` to observe behavior.

## Example folder

View the complete source code on GitHub: [src/2026-05-20 Basic Zephyr Application](#)

## Next topic

**Tomorrow, we will learn how to use Zephyr drivers and device binding properly.**

# Day 6 — Using Zephyr Drivers and Device Binding

## Goal

Learn how Zephyr drivers are accessed in code and how to bind to hardware devices.

## What you will learn

- The difference between `device_get_binding()` and `DEVICE_DT_GET()`
- How to use Zephyr device driver APIs
- How to check device readiness
- How to work with common peripherals like GPIO, I2C, and UART

## Overview

Zephyr provides a device model where peripherals are exposed as `struct device` objects. Your application can request these devices by name or via Device Tree macros.

## Driver access patterns

- `device_get_binding("UART_0")` for named devices
- `DEVICE_DT_GET(DT_NODELABEL(uart0))` for Device Tree-based access
- `device_is_ready(dev)` to verify the device

## Example workflow

1. Enable the peripheral in `prj.conf` and overlay
2. Get the device binding in `main()`

3. Use the driver API for operations like GPIO set/reset or I2C read/write

## Why this matters

Correct device binding ensures your code works with the actual hardware and is portable across boards.

## Practice tasks

1. Bind to a GPIO device and toggle an LED.
2. Bind to a UART device and send a test string.
3. Explore a sample driver API such as sensor, PWM, or I2C.

## Example folder

View the complete source code on GitHub: [src/2026-05-21\\_Zephyr\\_Drivers\\_and\\_Binding](#)

## Next topic

**Tomorrow, we will cover debugging and testing Zephyr applications.**

# Day 7 — Debugging and Testing Zephyr Applications

## Goal

Understand the Zephyr debugging and testing workflow for finding and fixing issues.

# What you will learn

- How to use logging and console output for debugging
- How to run Zephyr tests and sample validation
- The role of `west test`, `west debug`, and `west flash`
- How to interpret build and runtime errors

## Overview

Debugging Zephyr often involves enabling log output, using debugger tools, and running built-in test suites. These tools help you verify that your code and configuration are correct.

## Debug techniques

- `printf()` and `LOG_INF()` for runtime messages
- `west debug` to attach GDB to the target
- `west test` for Zephyr test cases
- `west build -t run` on simulation boards such as `native_posix`

## Why this matters

Good debugging practices accelerate learning, reduce frustration, and help you understand how your application interacts with Zephyr internals.

## Practice tasks

1. Add logging to an existing Zephyr app and watch the console output.
2. Run a sample on `native_posix` or a simulator with `west build -b native_posix`.
3. Review build warnings and fix one configuration or code issue.

## Example folder

View the complete source code on GitHub: [src/2026-05-22 Debugging and Testing](#)

## Next topic

**After this, you can continue with advanced Zephyr concepts such as Bluetooth, networking, and power management.**

# Day 8 — Threads and Synchronization

## Goal

Learn how to create and manage multiple threads in Zephyr and synchronize them using semaphores, mutexes, and message queues.

## What you will learn

- How to define and start threads using `K_THREAD_DEFINE`
- How to use `k_sem`, `k_mutex`, and `k_msgq` for synchronization
- Thread priorities and cooperative vs preemptive scheduling
- How to pass data between threads safely

## Overview

Zephyr is a preemptive RTOS. Each thread has a priority and a stack. Threads run concurrently and share resources, so synchronization primitives are needed to avoid race conditions.

## Key concepts

### Defining a thread:

```
K_THREAD_DEFINE(my_thread, 1024, my_thread_fn, NULL, NULL, NULL,  
5, 0, 0);
```

Arguments: name, stack size, entry function, arg1, arg2, arg3, priority, options, delay.

## Semaphore — signal between threads:

```
K_SEM_DEFINE(my_sem, 0, 1);  
k_sem_give(&my_sem);    // signal  
k_sem_take(&my_sem, K_FOREVER); // wait
```

## Mutex — protect shared data:

```
K_MUTEX_DEFINE(my_mutex);  
k_mutex_lock(&my_mutex, K_FOREVER);  
// critical section  
k_mutex_unlock(&my_mutex);
```

## Message queue — pass data between threads:

```
K_MSGQ_DEFINE(my_queue, sizeof(int), 10, 4);  
k_msgq_put(&my_queue, &value, K_NO_WAIT);  
k_msgq_get(&my_queue, &value, K_FOREVER);
```

## Thread priorities

- Lower number = higher priority
- Negative priorities are cooperative (must yield explicitly)
- Priorities 0–14 are preemptive by default

## Why this matters

Almost every real embedded application uses multiple threads: one for sensors, one for communication, one for UI. Understanding Zephyr threading is essential before building any non-trivial application.

## Practice tasks

1. Create two threads at different priorities and observe which runs first.
2. Use a semaphore to have thread A trigger thread B.
3. Pass an integer counter from a producer thread to a consumer thread via `k_msgq`.

## Example folder

View the complete source code on GitHub: [src/2026-05-23 Threads and Synchronization](#)

## Next topic

Tomorrow we explore GPIO configuration and interrupt-driven input handling.

---

# Day 9 — GPIO and Interrupts

## Goal

Learn how to configure GPIO pins in Zephyr for input and output, and how to respond to pin state changes using hardware interrupts.

## What you will learn

- How to get a GPIO device using the Device Tree
- How to configure a pin as output (LED) and input (button)
- How to register a GPIO interrupt callback
- How to debounce a button in software

## Overview

Zephyr's GPIO API abstracts hardware pin control across all supported boards. Pin configuration is done via the Device Tree, and the C API calls reference DT node labels.

## Key concepts

### Controlling an LED:

```
const struct gpio_dt_spec led = GPIO_DT_SPEC_GET(DT_ALIAS(led0),
gpis);
gpio_pin_configure_dt(&led, GPIO_OUTPUT_ACTIVE);
gpio_pin_set_dt(&led, 1);    // on
gpio_pin_set_dt(&led, 0);    // off
gpio_pin_toggle_dt(&led);    // toggle
```

### Reading a button:

```
const struct gpio_dt_spec btn = GPIO_DT_SPEC_GET(DT_ALIAS(sw0),
gpis);
```



```
gpio_pin_configure_dt(&btn, GPIO_INPUT);  
int val = gpio_pin_get_dt(&btn);
```

### Interrupt callback:

```
static struct gpio_callback btn_cb;  
  
void btn_handler(const struct device *dev, struct gpio_callback  
*cb, uint32_t pins)  
{  
    printk("Button pressed!\n");  
}  
  
gpio_pin_interrupt_configure_dt(&btn, GPIO_INT_EDGE_TO_ACTIVE);  
gpio_init_callback(&btn_cb, btn_handler, BIT(btn.pin));  
gpio_add_callback(btn.port, &btn_cb);
```

### nRF52840 DK aliases

- led0 through led3 — onboard LEDs
- sw0 through sw3 — onboard buttons

## Why this matters

GPIO is the foundation of hardware interaction. LEDs, buttons, relays, and many sensors are wired directly to GPIO pins. Interrupts let you respond instantly without polling.

## Practice tasks

1. Toggle led0 every 500 ms using gpio\_pin\_toggle\_dt.
2. Read sw0 in a loop and mirror its state to led1.
3. Use an interrupt on sw0 to toggle led0 on each press.

## Example folder

View the complete source code on GitHub: [src/  
2026-05-24 GPIO and Interrupts](#)

## Next topic

Tomorrow we learn how to define a custom PCB as a Zephyr board target.

---

# Day 10 — Custom PCB Board Definition

## Goal

Learn how to define a custom PCB as a named Zephyr board target so that your hardware is fully described by Device Tree and Kconfig rather than relying on a stock evaluation kit.

## What you will learn

- The file structure required for a custom board
- How to write a board-level `.dts` and `_defconfig`
- How to register GPIO, UART, I2C, and SPI peripherals for your PCB
- How to build and flash targeting your custom board name

## Overview

When you design your own PCB around a chip like the nRF52840, you create a board definition that tells Zephyr exactly which peripherals are wired up and how. This replaces the DK-specific assumptions in `nrf52840dk/nrf52840`.

## Required file structure

```
boards/
├── arm/
│   └── my_custom_board/
│       ├── Kconfig.board           ← board name declaration
│       ├── Kconfig.defconfig      ← default Kconfig symbols
│       └── board.cmake             ← flash runner (JLink, OpenOCD,
etc.)                               my_custom_board.dts ← hardware description
```

```
└─ my_custom_board.yaml ← board metadata
└─ my_custom_board_defconfig ← minimal kernel config
```

## my\_custom\_board.dts (skeleton)

```
/dts-v1/;
#include <nordic/nrf52840_qiaa.dtsi>

/ {
    model = "My Custom Board";
    compatible = "my-company,my-custom-board";

    chosen {
        zephyr,console = &uart0;
        zephyr,shell-uart = &uart0;
        zephyr,sram = &sram0;
        zephyr,flash = &flash0;
    };

    leds {
        compatible = "gpio-leds";
        led0: led_0 {
            gpios = <&gpio0 13 GPIO_ACTIVE_LOW>;
            label = "Status LED";
        };
    };

    aliases {
        led0 = &led0;
    };
};

&uart0 {
    status = "okay";
    compatible = "nordic,nrf-uarte";
    current-speed = <115200>;
    pinctrl-0 = <&uart0_default>;
    pinctrl-names = "default";
};

&i2c0 {
    status = "okay";
    pinctrl-0 = <&i2c0_default>;
    pinctrl-names = "default";
    clock-frequency = <I2C_BITRATE_FAST>;
};
```

## Kconfig.board

```
config BOARD_MY_CUSTOM_BOARD
    bool "My Custom Board"
    select SOC_NRF52840_QIAA
```

## board.cmake

```
board_runner_args(jlink "--device=nRF52840_xxAA" "--speed=4000")
include(${ZEPHYR_BASE}/boards/common/jlink.board.cmake)
```

## my\_custom\_board\_defconfig

```
CONFIG_SOC_SERIES_NRF52X=y
CONFIG_SOC_NRF52840_QIAA=y
CONFIG_BOARD_MY_CUSTOM_BOARD=y
CONFIG_SYS_CLOCK_HW_CYCLES_PER_SEC=64000000
CONFIG_GPIO=y
CONFIG_SERIAL=y
```

## Building for your custom board

Place the boards/ directory either: - Inside your application directory,  
or - In a separate module registered via west.yml

Then build with:

```
west build -b my_custom_board
```

## Pin control (nRF52840 specific)

nRF52840 uses pinctrl to assign peripheral functions to physical pins.  
Add a pinctrl block to your .dts:

```
&pinctrl {
    uart0_default: uart0_default {
        group1 {
            psels = <NRF_PSEL(UART_TX, 0, 6)>,
                  <NRF_PSEL(UART_RX, 0, 8)>;
        };
    };
    i2c0_default: i2c0_default {
        group1 {
            psels = <NRF_PSEL(TWIM_SDA, 0, 26)>,
                  <NRF_PSEL(TWIM_SCL, 0, 27)>;
        };
    };
};
```

```
};  
};
```

Replace pin numbers with those from your PCB schematic.

## Why this matters

Every production product needs a custom board definition. Keeping hardware description in DTS and Kconfig means your application code stays portable and board-agnostic.

## Practice tasks

1. Copy the `nrf52840dk` board directory from the Zephyr tree and rename it to `my_custom_board`.
2. Change the LED GPIO pin to match a pin on your schematic.
3. Build the example targeting `my_custom_board` — see [src/2026-05-25 Custom PCB Board Definition](#).

## Example folder

View the complete source code on GitHub: [src/2026-05-25 Custom PCB Board Definition](#)

## Next topic

Tomorrow we enable USB CDC-ACM so the board can communicate with a PC over USB.

---

# Day 11 — USB CDC-ACM Communication

## Goal

Enable USB communication on the nRF52840 so the board appears as a virtual serial port (COM port) on a connected PC.

## What you will learn

- What USB CDC-ACM is and why it is the simplest USB serial option
- How to enable the USB subsystem and CDC-ACM class in Kconfig
- How to redirect the Zephyr console to the USB virtual serial port
- How to write and read raw bytes over USB from application code

## Overview

CDC-ACM (Communications Device Class – Abstract Control Model) makes your microcontroller appear as a standard USB serial port. No custom driver is needed on Windows, Linux, or macOS. Data flows through the same `uart_poll_out` / `uart_fifo_read` API you already know.

## prj.conf settings

```
CONFIG_USB_DEVICE_STACK=y
CONFIG_USB_DEVICE_PRODUCT="My Zephyr USB Device"
CONFIG_USB_DEVICE_VID=0x2FE3
CONFIG_USB_DEVICE_PID=0x0100
CONFIG_USB_CDC_ACM=y
```

```
# Route the Zephyr console through USB CDC-ACM
CONFIG_UART_LINE_CTRL=y
CONFIG_USB_UART_CONSOLE=y
CONFIG_UART_CONSOLE=y
CONFIG_LOG=y
CONFIG_LOG_BACKEND_UART=y
```

## app.overlay — bind CDC-ACM to the console

```
/ {
    chosen {
        zephyr,console = &cdc_acm_uart0;
        zephyr,shell-uart = &cdc_acm_uart0;
    };
};

&zephyr_udc0 {
    cdc_acm_uart0: cdc_acm_uart0 {
        compatible = "zephyr,cdc-acm-uart";
        label = "CDC_ACM_0";
    };
};
```

## Initializing USB in application code

```
#include <zephyr/usb/usb_device.h>
#include <zephyr/drivers/uart.h>

const struct device *usb_uart =
DEVICE_DT_GET(DT_NODELABEL(cdc_acm_uart0));

void main(void)
{
    usb_enable(NULL);

    // Wait for DTR — the PC has opened the COM port
    uint32_t dtr = 0;
    while (!dtr) {
        uart_line_ctrl_get(usb_uart, UART_LINE_CTRL_DTR, &dtr);
        k_msleep(100);
    }

    printk("USB serial ready!\n");
}
```

## Reading incoming bytes

```
uint8_t buf[64];
int len = uart_fifo_read(usb_uart, buf, sizeof(buf));
```

## Connecting from a PC

- **Linux:** appears as /dev/ttyACM0
- **macOS:** appears as /dev/tty.usbmodem\*
- **Windows:** appears as COMx in Device Manager

Use any serial terminal (minicom, PuTTY, screen) at any baud rate (CDC-ACM ignores baud).

## Why this matters

USB serial is the most practical way to exchange data between an embedded device and a PC without dedicated RF hardware. It is the foundation for the CBOR-over-USB example in the next session.

## Practice tasks

1. Enable USB CDC-ACM and redirect `printf` output to the USB port.
2. Open a terminal on your PC and confirm you receive Zephyr log output.
3. Send a byte from the PC and echo it back from the firmware.

## Example folder

View the complete source code on GitHub: [src/2026-05-26\\_USB\\_CDC\\_ACM](#)

## Next topic

Tomorrow we encode structured data using CBOR and send it over this USB connection.

---

# Day 12 — CBOR Encoding over USB

## Goal

Use the CBOR (Concise Binary Object Representation) format to encode structured sensor data on the nRF52840 and transmit it over the USB CDC-ACM virtual serial port established in Day 11.



# What you will learn

- What CBOR is and why it is used in embedded systems
- How to enable the zcbor library in Zephyr
- How to encode a map (key-value structure) with zcbor
- How to send the encoded binary frame over USB and decode it on a PC

## Overview

CBOR (RFC 8949) is a binary data format similar to JSON but much more compact — ideal for bandwidth-constrained links like USB serial or BLE. Zephyr ships with zcbor, a code-generated CBOR library with a streaming encoder/decoder.

## Enable zcbor in prj.conf

```
CONFIG_ZCBOR=y
CONFIG_ZCBOR_STOP_ON_ERROR=y
```

## Encoding a simple map

```
#include <zcbor_encode.h>

uint8_t buf[64];
struct zcbor_state_t states[2];
zcbor_new_encode_state(states, ARRAY_SIZE(states), buf,
sizeof(buf), 1);

// Encode: {"temp": 2350, "hum": 65}
zcbor_map_start_encode(states, 2);
zcbor_tstr_put_lit(states, "temp");
zcbor_int32_put(states, 2350);           // 23.50 °C * 100
zcbor_tstr_put_lit(states, "hum");
zcbor_int32_put(states, 65);
zcbor_map_end_encode(states, 2);

size_t len = states[0].payload - buf;
// buf[0..len-1] contains the CBOR frame
```

## Sending over USB

```
for (size_t i = 0; i < len; i++) {
    uart_poll_out(usb_uart, buf[i]);
}
```

```
uart_poll_out(usb_uart, '\n'); // optional newline as frame
delimiter
```

## Framing strategy

Raw CBOR is binary — a newline delimiter works for simple cases but is not reliable if the data contains 0x0A. For robust framing, prefix each packet with a 2-byte length:

```
uint8_t header[2] = { (len >> 8) & 0xFF, len & 0xFF };
uart_poll_out(usb_uart, header[0]);
uart_poll_out(usb_uart, header[1]);
for (size_t i = 0; i < len; i++) {
    uart_poll_out(usb_uart, buf[i]);
}
```

## Decoding on the PC (Python)

```
import serial, cbor2

port = serial.Serial('/dev/ttyACM0', timeout=1)
raw = port.read(64) # read up to 64 bytes
data = cbor2.loads(raw) # {'temp': 2350, 'hum': 65}
print(data)
```

Install: `pip install pyserial cbor2`

## zcbor types reference

| Type        | Encode call                                      |
|-------------|--------------------------------------------------|
| Integer     | <code>zcbor_int32_put(state, val)</code>         |
| Text string | <code>zcbor_tstr_put_lit(state, "key")</code>    |
| Byte string | <code>zcbor_bstr_put_arr(state, buf, len)</code> |
| Boolean     | <code>zcbor_bool_put(state, true)</code>         |
| Array start | <code>zcbor_list_start_encode(state, max)</code> |
| Map start   | <code>zcbor_map_start_encode(state, max)</code>  |

## Why this matters

CBOR is the encoding of choice for CoAP, SenML, SUIT firmware updates, and many IoT protocols. Learning it now prepares you for Bluetooth, LwM2M, and cloud integrations later.

## Practice tasks

1. Encode a map with three fields: temp, hum, and a ts (timestamp as uint32).
2. Send it over USB every second and confirm receipt with the Python snippet above.
3. Extend to an array of 5 historical readings inside the same CBOR packet.

## Example folder

View the complete source code on GitHub: [src/2026-05-27\\_CBOR\\_over\\_USB](#)

## Next topic

Tomorrow we explore the Zephyr Shell subsystem to build an interactive command-line interface over UART or USB.

---

# Day 13 — Zephyr Shell Subsystem

## Goal

Use Zephyr's built-in Shell subsystem to add an interactive command-line interface to your firmware, accessible over UART or USB.

## What you will learn

- How to enable and configure the Zephyr Shell
- How to define custom shell commands with arguments
- How to nest commands into subcommand trees
- How to use the shell over USB CDC-ACM

# Overview

The Zephyr Shell gives you a fully featured CLI — command history, tab completion, help text — with minimal effort. It runs as a dedicated kernel thread and dispatches commands to your registered handlers.

## prj.conf settings

```
CONFIG_SHELL=y
CONFIG_SHELL_BACKEND_UART=y
CONFIG_UART_CONSOLE=y
```

To use the shell over USB instead:

```
CONFIG_SHELL=y
CONFIG_SHELL_BACKEND_SERIAL=y
CONFIG_USB_DEVICE_STACK=y
CONFIG_USB_CDC_ACM=y
CONFIG_UART_LINE_CTRL=y
```

## Registering a top-level command

```
#include <zephyr/shell/shell.h>

static int cmd_hello(const struct shell *sh, size_t argc, char
**argv)
{
    shell_print(sh, "Hello from shell!");
    return 0;
}

SHELL_CMD_REGISTER(hello, NULL, "Print a greeting", cmd_hello);
```

## Subcommands

```
SHELL_STATIC_SUBCMD_SET_CREATE(led_cmds,
    SHELL_CMD(on, NULL, "Turn LED on", cmd_led_on),
    SHELL_CMD(off, NULL, "Turn LED off", cmd_led_off),
    SHELL_SUBCMD_SET_END
);

SHELL_CMD_REGISTER(led, &led_cmds, "LED control commands", NULL);
```

Usage: led on, led off

## Reading arguments

```
static int cmd_set(const struct shell *sh, size_t argc, char
**argv)
{
    if (argc < 2) {
        shell_error(sh, "Usage: set <value>");
        return -EINVAL;
    }
    int val = atoi(argv[1]);
    shell_print(sh, "Value set to %d", val);
    return 0;
}
```

## Printing from outside the shell

Use `shell_fprintf` with a saved shell pointer, or use `printk` — both reach the console.

## Built-in shell commands

Zephyr's shell comes with built-in commands you get for free: - `help` — list all registered commands - `kernel uptime` — system uptime - `kernel reboot` - `cold` — restart the device - `log` — control log levels at runtime

## Why this matters

A shell interface is invaluable for calibration, diagnostics, and field testing. It lets you interact with a running device without rebuilding firmware.

## Practice tasks

1. Register a `ping` command that replies `pong`.
2. Add a `led` command with `on` and `off` subcommands that control `led0`.
3. Add a `read temp` subcommand that prints a simulated temperature value.

## Example folder

View the complete source code on GitHub: [src/2026-05-28\\_Zephyr\\_Shell](#)

## Next topic

Tomorrow we read data from a real I2C sensor and log it using everything learned so far.

---

# Day 14 — I2C Sensor Integration

## Goal

Read data from a real I2C sensor using Zephyr's sensor API, combining Device Tree configuration, driver binding, and the logging skills from previous days.

## What you will learn

- How to declare an I2C sensor in the Device Tree overlay
- How to fetch and use a sensor device via `DEVICE_DT_GET`
- How to read channels using `sensor_sample_fetch` and `sensor_channel_get`
- How to use the BME280 (temperature/humidity/pressure) as a worked example

## Overview

Zephyr provides a unified sensor API across all supported drivers. Once the sensor is declared in the Device Tree and the correct Kconfig driver is enabled, the application code is the same regardless of the specific chip.

### prj.conf

```
CONFIG_I2C=y
CONFIG_SENSOR=y
CONFIG_BME280=y          # replace with your sensor's Kconfig symbol
CONFIG_LOG=y
CONFIG_LOG_DEFAULT_LEVEL=3
```

## app.overlay — add sensor to I2C bus

```
&i2c0 {
    bme280@76 {
        compatible = "bosch,bme280";
        reg = <0x76>;          /* I2C address – check your sensor
datasheet */
        label = "BME280";
    };
};
```

The @76 suffix and reg value must match the sensor's 7-bit I2C address.

## Reading sensor data

```
#include <zephyr/drivers/sensor.h>

const struct device *bme = DEVICE_DT_GET_ANY(bosch_bme280);

void read_sensor(void)
{
    struct sensor_value temp, press, hum;

    sensor_sample_fetch(bme);
    sensor_channel_get(bme, SENSOR_CHAN_AMBIENT_TEMP, &temp);
    sensor_channel_get(bme, SENSOR_CHAN_PRESS, &press);
    sensor_channel_get(bme, SENSOR_CHAN_HUMIDITY, &hum);

    printk("Temp: %d.%02d C  Press: %d.%02d kPa  Hum: %d.%02d %%
\n",
           temp.val1, abs(temp.val2) / 10000,
           press.val1, abs(press.val2) / 10000,
           hum.val1, abs(hum.val2) / 10000);
}
```

## sensor\_value explained

sensor\_value is a fixed-point struct: - val1 — integer part - val2 — fractional part in millionths (μ)

So {23, 500000} = 23.5

## Common sensor Kconfig symbols

| Sensor                | Symbol          |
|-----------------------|-----------------|
| BME280 temp/hum/press | CONFIG_BME280=y |
| SHT3xD temp/hum       | CONFIG_SHT3XD=y |

| Sensor               | Symbol             |
|----------------------|--------------------|
| LIS2DH accelerometer | CONFIG_LIS2DH=y    |
| TMP116 temperature   | CONFIG_TMP116=y    |
| ICM-42688 IMU        | CONFIG_ICM42688P=y |

## I2C bus readiness check

Always verify the bus and device are ready before reading:

```
if (!device_is_ready(bme)) {  
    LOG_ERR("BME280 not ready – check wiring and I2C address");  
    return;  
}
```

## Why this matters

Almost every IoT product reads from sensors. The Zephyr sensor API means you can swap sensor chips by changing the overlay and Kconfig, with zero changes to your application logic.

## Practice tasks

1. Wire a BME280 to I2C pins on the nRF52840 DK (P0.26 = SDA, P0.27 = SCL).
2. Build and flash; confirm temperature readings in the log.
3. Combine with Day 12: encode the sensor reading as CBOR and send it over USB.

## Example folder

View the complete source code on GitHub: [src/2026-05-29 I2C Sensor Integration](#)

## Next topic

You now have the core building blocks. Next steps include BLE, power management, firmware updates via SUIT/MCUboot, and production hardening.

---



# Day 15 — KXTJ3-1057

## Accelerometer with USB Shell

### Goal

Implement a custom I2C driver for the KXTJ3-1057 3-axis accelerometer and expose it through an interactive Zephyr Shell over USB CDC-ACM.

### What you will learn

- How to write a minimal custom I2C driver when no Zephyr driver exists
- How to read raw accelerometer data and convert it to g-force
- How to combine USB CDC-ACM and the Zephyr Shell in one application
- How to build shell commands that read and configure hardware

### KXTJ3-1057 Overview

The KXTJ3-1057 (Kionix) is a low-power 3-axis MEMS accelerometer: - Interface: I2C (up to 400 kHz) - I2C address: **0x0E** (ADDR/SDO pin low) or **0x0F** (ADDR/SDO pin high) - Measurement ranges:  $\pm 2g$ ,  $\pm 4g$ ,  $\pm 8g$ ,  $\pm 16g$  - Output: 12-bit or 8-bit resolution - WHO\_AM\_I register: 0x0F  $\rightarrow$  value 0x35

### Key registers

| Register | Address | Description      |
|----------|---------|------------------|
| XOUT_L   | 0x06    | X-axis low byte  |
| XOUT_H   | 0x07    | X-axis high byte |
| YOUT_L   | 0x08    | Y-axis low byte  |
| YOUT_H   | 0x09    | Y-axis high byte |
| ZOUT_L   | 0x0A    | Z-axis low byte  |
| ZOUT_H   | 0x0B    | Z-axis high byte |
| WHO_AM_I | 0x0F    | Device ID (0x35) |

| Register  | Address | Description                    |
|-----------|---------|--------------------------------|
| CTRL_REG1 | 0x1B    | Power/range/resolution control |

## CTRL\_REG1 bit map

Bit 7: PC1 – 1 = operating mode, 0 = standby  
 Bit 6: RES – 1 = 12-bit, 0 = 8-bit  
 Bit 4: GSEL1 – range select MSB  
 Bit 3: GSEL0 – range select LSB  
 00 =  $\pm 2g$ , 01 =  $\pm 4g$ , 10 =  $\pm 8g$  (14-bit), 11 =  $\pm 8g$

## Hardware wiring (nRF52840 DK)

| KXTJ3-1057 pin | nRF52840 DK pin           |
|----------------|---------------------------|
| VDD            | 3.3V                      |
| GND            | GND                       |
| SDA            | P0.26                     |
| SCL            | P0.27                     |
| ADDR           | GND ( $\rightarrow$ 0x0E) |

## Project structure

```

src/2026-05-30_KXTJ3_Accel_USB_Shell/
├─ CMakeLists.txt
├─ prj.conf
├─ app.overlay
├─ drivers/
│   └─ kxtj3/
│       ├── kxtj3.h      ← register map and API
│       └─ kxtj3.c      ← I2C read/write implementation
└─ src/
    └─ main.c           ← USB init + shell commands
  
```

## prj.conf

```

CONFIG_I2C=y
CONFIG_USB_DEVICE_STACK=y
CONFIG_USB_CDC_ACM=y
CONFIG_UART_LINE_CTRL=y
CONFIG_SHELL=y
CONFIG_SHELL_BACKEND_SERIAL=y
  
```

```
CONFIG_LOG=y
CONFIG_LOG_DEFAULT_LEVEL=3
```

## Shell commands implemented

| Command                                      | Description                          |
|----------------------------------------------|--------------------------------------|
| <code>accel whoami</code>                    | Read WHO_AM_I register (expect 0x35) |
| <code>accel read</code>                      | Print X, Y, Z in mg                  |
| <code>accel stream &lt;n&gt;</code>          | Print n readings at 200 ms intervals |
| <code>accel range &lt;2\ 4\ 8\ 16&gt;</code> | Set measurement range                |

## Converting raw counts to mg

The 12-bit result is left-justified in a 16-bit register pair. Shift right by 4 to get the signed 12-bit value, then scale:

```
int16_t raw = (int16_t)((high << 8) | low) >> 4;
int32_t mg = (raw * 1000) / counts_per_g;
// counts_per_g = 1024 for ±2g, 512 for ±4g, 256 for ±8g
```

## Why this matters

Many sensors used in production products are not supported by Zephyr's upstream drivers. Writing a thin I2C wrapper is a core embedded skill. The shell interface lets you validate sensor output during bring-up without rebuilding firmware.

## Practice tasks

1. Verify WHO\_AM\_I returns 0x35 before reading data.
2. Hold the board still and confirm Z reads close to 1000 mg (1g).
3. Try `accel range 4` and confirm the readings change scale.
4. Extend `accel stream` to also encode data as CBOR (combine with Day 12).

## Example folder

View the complete source code on GitHub: [src/2026-05-30\\_KXTJ3\\_Accel\\_USB\\_Shell](#)

## Next topic

Future topics: BLE peripheral advertising, MCUboot firmware updates, low-power sleep modes.

---

# Day 16 — KXTJ3-1057 Accelerometer with CBOR over USB

## Goal

Extend the KXTJ3-1057 driver from Day 15 to encode accelerometer readings as CBOR and transmit them over USB CDC-ACM. Add shell commands to control the output format and a Python script to receive, decode, and display the data on a PC.

## What you will learn

- How to combine zcbor encoding with real sensor data
- How to design a simple binary protocol using CBOR maps
- How to trigger CBOR output from an interactive shell
- How to write a Python receiver to decode and display the sensor stream

## CBOR frame format

Each transmission is a self-contained CBOR map:

```
{  
  "seq":  <uint32>,    sequence number  
  "x":    <int32>,      X-axis in mg
```

```

    "y":    <int32>,    Y-axis in mg
    "z":    <int32>,    Z-axis in mg
    "rng":  <uint8>     current range in g (2, 4, 8, or 16)
}

```

The frame is followed by a newline (0x0A) as a simple delimiter so the Python receiver can use `readline()` to receive complete packets.

## Encoded size

A typical frame encodes to ~25–30 bytes — well within a single USB packet.

## Shell commands

| Command                             | Description                            |
|-------------------------------------|----------------------------------------|
| <code>accel whoami</code>           | Verify WHO_AM_I register               |
| <code>accel read</code>             | Print one reading as plain text        |
| <code>accel cbor</code>             | Send one CBOR frame over USB           |
| <code>accel stream &lt;n&gt;</code> | Send n CBOR frames at 200 ms intervals |
| <code>accel range &lt;g&gt;</code>  | Set measurement range (2/4/8/16)       |

## Python decoder (`tools/decode_accel.py`)

The script connects to the USB serial port, reads CBOR frames, and prints a formatted table. Optional `--plot` flag renders a live scrolling graph using matplotlib.

```

# Install dependencies
pip install pyserial cbor2 matplotlib

# Basic usage
python3 tools/decode_accel.py --port /dev/ttyACM0

# With live plot
python3 tools/decode_accel.py --port /dev/ttyACM0 --plot

```

## Example output

| SEQ | X (mg) | Y (mg) | Z (mg) | Range |
|-----|--------|--------|--------|-------|
| 0   | -12    | 34     | 998    | ±2g   |

|   |     |    |     |     |
|---|-----|----|-----|-----|
| 1 | -13 | 33 | 997 | ±2g |
| 2 | -11 | 35 | 999 | ±2g |

## Why this matters

Sending structured binary data (CBOR) instead of plain text strings is essential for production firmware — it is compact, self-describing, and easy to parse on any platform. This pattern is used in CoAP, SenML, LwM2M, and Bluetooth GATT descriptors.

## Practice tasks

1. Flash and run `accel_cbor` — confirm the Python script decodes correctly.
2. Run `accel_stream 50` and observe the live plot.
3. Shake the board and confirm all three axes respond.
4. Switch to `accel_range 4` mid-stream and confirm the mg values change scale.

## Example folder

View the complete source code on GitHub: [src/2026-05-31\\_KXTJ3\\_CBOR\\_USB](#)

## Next topic

Future topics: BLE peripheral advertising sensor data, MCUboot OTA updates, low-power sleep with sensor wake-on-motion.

---

# Day 17 — KXTJ3-1057 CBOR Accelerometer on Custom PCB

## Goal

Deploy the KXTJ3-1057 CBOR-over-USB application from Day 16 onto a custom PCB board target instead of the nRF52840 DK. This teaches you how to move from a development kit to a production board definition.

## What you will learn

- How to define a custom board that includes the KXTJ3-1057 as a named DTS node
- How to assign specific PCB pins in the Device Tree and pinctrl
- How to build and flash the full CBOR accelerometer app targeting `my_custom_board`
- How the board definition and application code stay cleanly separated

## What changes compared to Day 16

| Aspect          | Day 16 (DK)           | Day 17 (Custom PCB)          |
|-----------------|-----------------------|------------------------------|
| Build target    | nrf52840dk/nrf52840   | my_custom_board              |
| Pin assignments | DK defaults           | Your PCB schematic           |
| KXTJ3 DTS node  | app.overlay           | board .dts file              |
| USB D+/D- pins  | DK defaults           | PCB-specific pinctrl         |
| app.overlay     | adds KXTJ3 + USB node | only adds CDC-ACM shell node |

## Custom PCB assumptions

This example assumes an nRF52840-based custom PCB with:

| Signal    | nRF52840 GPIO | Reason                        |
|-----------|---------------|-------------------------------|
| I2C SDA   | P0.04         | Chosen to avoid UART conflict |
| I2C SCL   | P0.05         | Adjacent to SDA               |
| UART TX   | P0.06         | Debug UART to USB-serial chip |
| UART RX   | P0.08         |                               |
| LED       | P1.10         | Active-low status LED         |
| KXTJ3 INT | P0.11         | Optional data-ready interrupt |

**Update these to match your actual schematic before building.**

## Board definition highlights

### my\_custom\_board.dts — KXTJ3 defined at board level

```
&i2c0 {
    status = "okay";
    kxtj3: kxtj3@e {
        compatible = "i2c-device";
        reg = <0x0e>;
        label = "KXTJ3";
    };
};
```

Because the KXTJ3 is soldered to the PCB, it belongs in the board .dts, not in the application overlay. The application overlay only adds the USB CDC-ACM shell node.

### app.overlay — only USB CDC-ACM

```
/ {
    chosen {
        zephyr,console = &cdc_acm_uart0;
        zephyr,shell-uart = &cdc_acm_uart0;
    };
};

&zephyr_udc0 {
    cdc_acm_uart0: cdc_acm_uart0 {
        compatible = "zephyr,cdc-acm-uart";
        label = "CDC_ACM_0";
    };
};
```



## Building for the custom board

```
# CMakeLists.txt sets BOARD_ROOT to pick up the local boards/  
directory  
west build -b my_custom_board  
west flash
```

## Python decoder (unchanged from Day 16)

```
pip install pyserial cbor2 matplotlib  
python3 tools/decode_accel.py --port /dev/ttyACM0 --plot
```

## Migration checklist

When moving any application from a DK to a custom board: - [ ] Copy the closest upstream board DTS as a starting point - [ ] Update all pin numbers to match your schematic - [ ] Move any always-present peripherals (sensors, displays) from overlay to board DTS - [ ] Keep application-specific overlays for things that could vary (USB, console routing) - [ ] Verify west build succeeds with -b my\_custom\_board - [ ] Check build/zephyr/zephyr.dts to confirm pin assignments are resolved correctly

## Why this matters

Every shipped product has a custom board definition. Learning to separate board hardware description from application logic is the key skill that makes your firmware portable and maintainable across board revisions.

## Example folder

View the complete source code on GitHub: [src/2026-06-01\\_KXTJ3\\_CBOR\\_Custom\\_PCB](#)

## Next topic

Future topics: BLE peripheral advertising, MCUboot OTA firmware updates, low-power wake-on-motion using KXTJ3 interrupt pin.

---

# Day 18 — BLE Peripheral Advertising

## Goal

Set up the nRF52840 as a Bluetooth Low Energy (BLE) peripheral that advertises a device name and a custom manufacturer data payload, connectable by a phone or PC.

## What you will learn

- How to enable and initialize the Zephyr BLE stack
- How to construct advertising and scan response data
- How to start and stop advertising
- How to handle connect and disconnect events
- How to read the device address and advertised name with a phone

## Overview

BLE advertising is the mechanism by which a peripheral announces its presence. The nRF52840 has a built-in radio that Zephyr drives through the `bt` API. No external hardware is required — just the SoC and a power supply.

## prj.conf

```
CONFIG_BT=y
CONFIG_BT_PERIPHERAL=y
CONFIG_BT_DEVICE_NAME="MyZephyrDevice"
CONFIG_BT_DEVICE_APPEARANCE=0
CONFIG_LOG=y
```

## Advertising data structure

```
static const struct bt_data ad[] = {
    BT_DATA_BYTES(BT_DATA_FLAGS, BT_LE_AD_GENERAL |
BT_LE_AD_NO_BREDR),
    BT_DATA(BT_DATA_NAME_COMPLETE, CONFIG_BT_DEVICE_NAME,
sizeof(CONFIG_BT_DEVICE_NAME) - 1),
};
```

```

    /* Manufacturer-specific data in scan response */
    static const uint8_t manuf_data[] = { 0xFF, 0x07, 0x01, 0x02,
0x03 };
    static const struct bt_data sd[] = {
        BT_DATA(BT_DATA_MANUFACTURER_DATA, manuf_data,
sizeof(manuf_data)),
    };

```

## Starting advertising

```

    bt_enable(NULL);    /* initialise the BLE stack */

    bt_le_adv_start(BT_LE_ADV_CONN, ad, ARRAY_SIZE(ad), sd,
ARRAY_SIZE(sd));

```

BT\_LE\_ADV\_CONN = connectable undirected advertising at 100 ms interval.

## Connection callbacks

```

    static void connected(struct bt_conn *conn, uint8_t err) {
        printk("Connected\n");
    }
    static void disconnected(struct bt_conn *conn, uint8_t reason) {
        printk("Disconnected (reason %u)\n", reason);
        bt_le_adv_start(BT_LE_ADV_CONN, ad, ARRAY_SIZE(ad), sd,
ARRAY_SIZE(sd));
    }

    BT_CONN_CB_DEFINE(conn_callbacks) = {
        .connected = connected,
        .disconnected = disconnected,
    };

```

## Verifying on a phone

- **Android:** nRF Connect app → Scanner tab → find “MyZephyrDevice”
- **iOS:** LightBlue app → Peripherals tab → find “MyZephyrDevice”
- **Linux:** bluetoothctl scan on → look for the device name

## Why this matters

BLE advertising is the entry point to all BLE applications — sensor beacons, wearables, and IoT devices all start here. Once advertising works, you can add GATT services to expose data to connected clients.

## Practice tasks

1. Flash and scan with nRF Connect — confirm the device name appears.
2. Change `CONFIG_BT_DEVICE_NAME` and rebuild — confirm the new name.
3. Add a second byte to `manuf_data` containing a counter that increments each time the device boots (combine with NVS from Day 22).

## Example folder

View the complete source code on GitHub: [src/2026-06-02\\_BLE\\_Peripheral\\_Advertising](#)

## Next topic

Tomorrow we look at SPI communication to interface with external peripherals.

---

# Day 19 — SPI Communication

## Goal

Use Zephyr's SPI API to communicate with an external SPI device — reading and writing registers over a full-duplex bus, with a practical example using a W25Q flash chip (widely available and cheap).

# What you will learn

- How to configure an SPI bus and chip-select in the Device Tree
- How to perform SPI read and write transfers using `spi_transceive`
- How to read a JEDEC ID from an SPI flash as a “hello world” for SPI
- How to write and read back a page of data

## Overview

SPI is a synchronous, full-duplex 4-wire bus: MOSI, MISO, SCK, and CS. Zephyr’s `spi_transceive` API sends and receives simultaneously. The Device Tree declares which SPI controller and CS pin to use.

### app.overlay

```
&spi1 {
    status = "okay";
    cs-gpios = <&gpio0 17 GPIO_ACTIVE_LOW>;

    w25q: w25q@0 {
        compatible = "jedec,spi-nor";
        reg = <0>;
        spi-max-frequency = <80000000>;
        size = <0x800000>; /* 8 Mbit */
        has-dpd;
        t-enter-dpd = <3000>;
        t-exit-dpd = <30000>;
    };
};
```

### prj.conf

```
CONFIG_SPI=y
CONFIG_FLASH=y
CONFIG_FLASH_MAP=y
CONFIG_LOG=y
```

## Reading the JEDEC ID (raw SPI transaction)

```
#include <zephyr/drivers/spi.h>

static const struct device *spi =
DEVICE_DT_GET(DT_NODELABEL(spi1));

static const struct spi_config cfg = {
    .frequency = 8000000,
```

```

        .operation = SPI_OP_MODE_MASTER | SPI_WORD_SET(8),
        .cs = SPI_CS_CONTROL_INIT(DT_NODELABEL(w25q), 2),
    };

    void read_jedec_id(void)
    {
        uint8_t tx_buf[] = { 0x9F };    /* JEDEC Read ID command */
        uint8_t rx_buf[4] = { 0 };

        struct spi_buf tx[] = {{ .buf = tx_buf, .len =
sizeof(tx_buf) }};
        struct spi_buf rx[] = {{ .buf = rx_buf, .len =
sizeof(rx_buf) }};
        struct spi_buf_set tx_set = { .buffers = tx, .count = 1 };
        struct spi_buf_set rx_set = { .buffers = rx, .count = 1 };

        spi_transceive(spi, &cfg, &tx_set, &rx_set);
        /* rx_buf[1..3] = Manufacturer, Memory Type, Capacity */
        printk("JEDEC ID: %02X %02X %02X\n", rx_buf[1], rx_buf[2],
rx_buf[3]);
    }

```

## Using the Zephyr Flash API (higher level)

```

#include <zephyr/drivers/flash.h>

const struct device *flash_dev =
DEVICE_DT_GET(DT_NODELABEL(w25q));
uint8_t data[256];

flash_read(flash_dev, 0, data, sizeof(data));
flash_erase(flash_dev, 0, 4096);    /* erase one sector
*/
flash_write(flash_dev, 0, data, sizeof(data));

```

## nRF52840 DK SPI wiring (W25Q flash module)

| Signal | nRF52840 DK pin |
|--------|-----------------|
| MOSI   | P0.20           |
| MISO   | P0.21           |
| SCK    | P0.19           |
| CS     | P0.17           |
| VCC    | 3.3V            |
| GND    | GND             |

## Why this matters

SPI is used for displays, flash memory, sensors (IMUs, barometers), DACs, and ADCs. The same `spi_transceive` API works for all of them.

## Practice tasks

1. Wire a W25Q16 and confirm the JEDEC ID reads EF 40 15.
2. Erase sector 0, write your name as ASCII, read it back.
3. Try increasing the SPI clock frequency and confirm communication stays stable.

## Example folder

View the complete source code on GitHub: [src/2026-06-03\\_SPI\\_Communication](#)

## Next topic

Tomorrow we read analog signals using the nRF52840's built-in ADC.

---

# Day 20 — ADC Reading

## Goal

Use the nRF52840's built-in 12-bit SAR ADC (SAADC) to read an analog voltage, convert it to millivolts, and log the result periodically.

## What you will learn

- How to declare an ADC channel in the Device Tree
- How to configure gain, reference, and acquisition time
- How to perform a single-shot ADC reading with `adc_read`
- How to convert raw counts to millivolts using `adc_raw_to_millivolts`

- Practical uses: battery monitoring, potentiometer, analog sensor

## Overview

The nRF52840 SAADC has 8 channels, 8/10/12-bit resolution, configurable gain and reference. Zephyr abstracts this through the `adc_` API.

### app.overlay

```
/ {
    zephyr,user {
        /* AIN0 maps to P0.02 on nRF52840 */
        io-channels = <&adc 0>;
    };
};

&adc {
    status = "okay";
    #address-cells = <1>;
    #size-cells = <0>;

    channel@0 {
        reg = <0>;
        zephyr,gain          = "ADC_GAIN_1_6";
        zephyr,reference      = "ADC_REF_INTERNAL";
        zephyr,acquisition-time = <ADC_ACQ_TIME_DEFAULT>;
        zephyr,input-positive = <NRF_SAADC_AIN0>;    /* P0.02 */
    };
};
```

### prj.conf

```
CONFIG_ADC=y
CONFIG_LOG=y
```

## Reading the ADC

```
#include <zephyr/drivers/adc.h>

#define ADC_NODE    DT_PATH(zephyr_user)
#define ADC_CHANNEL DT_IO_CHANNELS_INPUT_BY_IDX(ADC_NODE, 0)

static const struct adc_dt_spec adc_ch =
ADC_DT_SPEC_GET(ADC_NODE);

void read_adc(void)
```



```

{
    int16_t raw;
    struct adc_sequence seq = {
        .buffer      = &raw,
        .buffer_size = sizeof(raw),
    };

    adc_sequence_init_dt(&adc_ch, &seq);
    adc_read(adc_ch.dev, &seq);

    int32_t mv = raw;
    adc_raw_to_millivolts_dt(&adc_ch, &mv);
    printk("ADC: raw=%d  voltage=%d mV\n", raw, mv);
}

```

## nRF52840 SAADC AIN mapping

| AIN  | GPIO  | Notes                                        |
|------|-------|----------------------------------------------|
| AIN0 | P0.02 | Also used for UART RTS on DK — use with care |
| AIN1 | P0.03 | Also used for UART CTS                       |
| AIN2 | P0.04 | Free on most DK configs                      |
| AIN3 | P0.05 | Free on most DK configs                      |
| AIN4 | P0.28 |                                              |
| AIN5 | P0.29 |                                              |
| AIN6 | P0.30 |                                              |
| AIN7 | P0.31 |                                              |

## Measuring battery voltage (resistor divider)

Wire a 10 M $\Omega$  / 4.7 M $\Omega$  divider from VBAT to AIN, GND. Scale factor =  $(10 + 4.7) / 4.7 \approx 3.13$ .

```
int32_t vbat_mv = mv * 147 / 47;  /* fixed-point scale */
```

## Why this matters

ADC is essential for battery monitoring, analog sensors (light, temperature via NTC, pressure), and potentiometer-based UIs. Understanding gain and reference settings prevents measurement errors.

## Practice tasks

1. Wire a 10 k $\Omega$  potentiometer (wiper to AIN2/P0.04) and confirm readings sweep 0–3600 mV as you turn it.
2. Add a second ADC channel on AIN3 and log both simultaneously.
3. Average 8 consecutive readings to reduce noise.

## Example folder

View the complete source code on GitHub: [src/2026-06-04\\_ADC\\_Reading](#)

## Next topic

Tomorrow we use PWM to smoothly dim an LED and control servo position.

---

# Day 21 — PWM LED Dimming

## Goal

Use Zephyr's PWM API to smoothly dim and brighten an LED using a breathing animation, and understand how to map PWM channels in the Device Tree.

## What you will learn

- How to declare a PWM channel in the Device Tree using `pwm-leds`
- How to set duty cycle with `pwm_set_dt`
- How to implement a smooth breathing effect
- How PWM period and pulse width relate to duty cycle and frequency

# Overview

PWM (Pulse Width Modulation) rapidly switches a pin on and off. The ratio of on-time (pulse) to total period (period) is the duty cycle. At 1 kHz, the eye and motor cannot follow individual pulses — they perceive an average.

## app.overlay

```
/ {
    pwmleds {
        compatible = "pwm-leds";
        pwm_led0: pwm_led_0 {
            pwms = <&pwm0 0 PWM_MSEC(20) PWM_POLARITY_NORMAL>;
            label = "PWM LED";
        };
    };

    aliases {
        pwm-led0 = &pwm_led0;
    };
};

&pwm0 {
    status = "okay";
    pinctrl-0 = <&pwm0_default>;
    pinctrl-names = "default";
};

&pinctrl {
    pwm0_default: pwm0_default {
        group1 {
            /* LED1 on nRF52840 DK = P0.13 */
            psels = <NRF_PSEL(PWM_OUT0, 0, 13)>;
        };
    };
};
```

## prj.conf

```
CONFIG_PWM=y
CONFIG_LOG=y
```

## Setting duty cycle

```
#include <zephyr/drivers/pwm.h>
```

```
static const struct pwm_dt_spec pwm_led =
```

```

PWM_DT_SPEC_GET(DT_ALIAS(pwm_led0));

/* 50% duty cycle */
pwm_set_dt(&pwm_led, pwm_led.period, pwm_led.period / 2);

/* 0% – fully off */
pwm_set_dt(&pwm_led, pwm_led.period, 0);

/* 100% – fully on */
pwm_set_dt(&pwm_led, pwm_led.period, pwm_led.period);

```

## Breathing effect

```

/* Ramp pulse from 0 to period in 50 steps, then back down */
uint32_t period = pwm_led.period;
int step = period / 50;

while (1) {
    for (uint32_t pulse = 0; pulse <= period; pulse += step) {
        pwm_set_dt(&pwm_led, period, pulse);
        k_msleep(20);
    }
    for (uint32_t pulse = period; pulse > 0; pulse -= step) {
        pwm_set_dt(&pwm_led, period, pulse);
        k_msleep(20);
    }
}

```

## Servo control

PWM is also used for hobby servos (50 Hz, 1–2 ms pulse):

```

/* Neutral position: 1.5 ms pulse at 50 Hz (20 ms period) */
pwm_set_dt(&servo, PWM_MSEC(20), PWM_USEC(1500));
/* Full left: 1 ms */
pwm_set_dt(&servo, PWM_MSEC(20), PWM_USEC(1000));
/* Full right: 2 ms */
pwm_set_dt(&servo, PWM_MSEC(20), PWM_USEC(2000));

```

## Why this matters

PWM is used everywhere: LED brightness control, motor speed, servo position, buzzer tones, and switching power supplies. Zephyr’s Device Tree mapping means you change a pin number in the overlay, not in the application code.

## Practice tasks

1. Flash and observe the breathing LED on LED1 (P0.13).
2. Change the step size and sleep delay to speed up or slow down the effect.
3. Wire a buzzer to a PWM pin and play a 1 kHz tone.

## Example folder

View the complete source code on GitHub: [src/2026-06-05\\_PWM\\_LED\\_Dimming](#)

## Next topic

Tomorrow we store configuration data in internal flash using Zephyr's NVS.

---

# Day 22 — NVS Flash Storage

## Goal

Use Zephyr's NVS (Non-Volatile Storage) subsystem to persist configuration and state data across power cycles in the nRF52840's internal flash.

## What you will learn

- What NVS is and how it differs from raw flash writes
- How to define an NVS partition in the flash map
- How to write, read, and delete key-value pairs with `nvs_write / nvs_read`
- A practical example: counting boots and storing a device config struct

# Overview

NVS stores arbitrary binary data as key-value pairs in a flash sector. It handles wear leveling automatically — writes spread across the sector so no single location wears out. Data survives resets and power loss.

## Flash partition (app.overlay)

```
/ {
    fstab {
        compatible = "zephyr,fstab";
    };
};

&flash0 {
    partitions {
        compatible = "fixed-partitions";
        #address-cells = <1>;
        #size-cells = <1>;

        boot_partition: partition@0 {
            label = "mcuboot";
            reg = <0x00000000 0x0000C000>;
        };
        slot0_partition: partition@c000 {
            label = "image-0";
            reg = <0x0000C000 0x00067000>;
        };
        storage_partition: partition@73000 {
            label = "storage";
            reg = <0x00073000 0x0000D000>;
        };
    };
};
```

## prj.conf

```
CONFIG_NVS=y
CONFIG_FLASH=y
CONFIG_FLASH_PAGE_LAYOUT=y
CONFIG_FLASH_MAP=y
CONFIG_LOG=y
```

## NVS key IDs (define your own)

```
#define NVS_ID_BOOT_COUNT 1
#define NVS_ID_DEVICE_CFG 2
```

## Initialising the NVS filesystem

```
#include <zephyr/fs/nvs.h>
#include <zephyr/storage/flash_map.h>

static struct nvs_fs fs;

void nvs_init_storage(void)
{
    const struct flash_area *fa;
    flash_area_open(FIXED_PARTITION_ID(storage_partition), &fa);

    fs.flash_device = flash_area_get_device(fa);
    fs.offset        = fa->fa_off;
    fs.sector_size   = 4096;
    fs.sector_count  = 4;

    nvs_mount(&fs);
}
```

## Writing and reading a boot counter

```
uint32_t boot_count = 0;

/* Read existing value (returns -ENOENT on first boot) */
nvs_read(&fs, NVS_ID_BOOT_COUNT, &boot_count, sizeof(boot_count));
boot_count++;
nvs_write(&fs, NVS_ID_BOOT_COUNT, &boot_count,
sizeof(boot_count));
printk("Boot count: %u\n", boot_count);
```

## Storing a config struct

```
struct device_cfg {
    uint8_t  range_g;
    uint16_t sample_rate_ms;
    char     name[16];
};

struct device_cfg cfg = { .range_g = 4, .sample_rate_ms =
200, .name = "sensor1" };
nvs_write(&fs, NVS_ID_DEVICE_CFG, &cfg, sizeof(cfg));

/* Later, on next boot */
struct device_cfg loaded = { 0 };
if (nvs_read(&fs, NVS_ID_DEVICE_CFG, &loaded, sizeof(loaded)) >
0) {
    printk("Loaded config: range=%dg rate=%dms name=%s\n",
```

```
        loaded.range_g, loaded.sample_rate_ms, loaded.name);  
    }
```

## Deleting an entry

```
nvs_delete(&fs, NVS_ID_DEVICE_CFG);
```

## Why this matters

Any real product needs to remember settings across power cycles: calibration values, user preferences, device identity, error logs, and usage counters. NVS is the simplest way to do this in Zephyr without a file system.

## Practice tasks

1. Flash and open a serial terminal — confirm boot count increments each reset.
2. Store and reload a config struct; verify all fields survive a power cycle.
3. Combine with the Shell (Day 13): add `config save` and `config load` commands.

## Example folder

View the complete source code on GitHub: [src/2026-06-06 NVS Flash Storage](#)

## Next topic

Tomorrow we explore work queues and kernel timers for deferred and periodic tasks.

---



# Day 23 — Work Queues and Kernel Timers

## Goal

Learn how to schedule deferred and periodic work in Zephyr using kernel timers and work queues, without blocking threads or polling in a tight loop.

## What you will learn

- How `k_timer` fires a callback at a set interval
- How `k_work` defers processing out of ISR / timer context
- How `k_work_delayable` schedules work after a delay
- How to create a dedicated work queue thread for heavy background tasks

## Overview

Timers and work queues are Zephyr's core tools for time-driven logic. A timer fires in interrupt context — keep the callback short. Hand off real work to a `k_work` item that runs in a thread.

## One-shot and periodic timers

```
static void timer_cb(struct k_timer *timer) {
    /* runs in ISR context – keep it short */
    printk("tick\n");
}

K_TIMER_DEFINE(my_timer, timer_cb, NULL);

/* Start: fire once after 2 s */
k_timer_start(&my_timer, K_SECONDS(2), K_NO_WAIT);

/* Start: fire every 500 ms after an initial 1 s delay */
k_timer_start(&my_timer, K_SECONDS(1), K_MSEC(500));

/* Stop */
k_timer_stop(&my_timer);
```

## Submitting work from a timer

```
static struct k_work my_work;

static void work_handler(struct k_work *work) {
    /* runs in system work queue thread – safe to call kernel APIs */
    LOG_INF("Work executed at uptime %lld ms", k_uptime_get());
}

static void timer_cb(struct k_timer *timer) {
    k_work_submit(&my_work); /* safe to call from ISR */
}

K_TIMER_DEFINE(my_timer, timer_cb, NULL);

void main(void) {
    k_work_init(&my_work, work_handler);
    k_timer_start(&my_timer, K_NO_WAIT, K_SECONDS(1));
}
```

## Delayable work (self-rescheduling)

```
static struct k_work_delayable dwork;

static void dwork_handler(struct k_work *work) {
    LOG_INF("Deferred work");
    /* reschedule itself */
    k_work_reschedule(&dwork, K_MSEC(500));
}

void main(void) {
    k_work_init_delayable(&dwork, dwork_handler);
    k_work_schedule(&dwork, K_NO_WAIT);
}
```

## Custom work queue (dedicated thread)

```
K_THREAD_STACK_DEFINE(my_wq_stack, 1024);
static struct k_work_q my_wq;

k_work_queue_start(&my_wq, my_wq_stack,
                  K_THREAD_STACK_SIZEOF(my_wq_stack),
                  K_PRIO_PREEMPT(5), NULL);

/* Submit to your queue instead of the system queue */
k_work_submit_to_queue(&my_wq, &my_work);
```

Use a dedicated queue when work is heavy, slow, or must not block the system work queue that other Zephyr subsystems rely on.

## Checking timer remaining time

```
k_ticks_t remaining = k_timer_remaining_ticks(&my_timer);  
int64_t ms          = k_ticks_to_ms_floor64(remaining);
```

## Why this matters

Work queues are used throughout Zephyr internals — the BLE stack, USB stack, and sensor trigger system all post work items. Understanding them lets you build responsive, non-blocking firmware without complex thread management.

## Practice tasks

1. Create a 1 Hz timer that submits work to toggle led0.
2. Add a `k_work_delayable` that prints uptime every 3 s without a timer.
3. Create a dedicated work queue and move heavy logging there.

## Example folder

View the complete source code on GitHub: [src/2026-06-07 Work Queues and Timers](#)

## Next topic

Tomorrow we configure Zephyr's power management to put the nRF52840 into low-power sleep between work items.

---

# Day 24 — Power Management and Sleep

## Goal

Configure Zephyr's power management subsystem to put the nRF52840 into low-power sleep states between tasks, dramatically reducing current consumption.

## What you will learn

- Zephyr's power management states and how the PM subsystem works
- How to enable automatic idle sleep with `CONFIG_PM`
- How to use `k_sleep` and `k_msleep` to hint the kernel to sleep
- How to measure current draw with and without PM enabled
- How to use the nRF52840's System OFF mode for ultra-low power

## Overview

The nRF52840 has several power states: | State | Current | Wake source | |——|——|———| | Active (CPU running) | ~4 mA | — | | WFI (Wait For Interrupt) | ~1.5 mA | Any IRQ | | System ON sleep (radio off) | ~2–4 µA | GPIO, timer, RTC | | System OFF | ~0.4 µA | GPIO DETECT, RESETN |

Zephyr's PM subsystem selects the deepest safe sleep state automatically whenever all threads are blocked or sleeping.

### prj.conf — enable automatic power management

```
CONFIG_PM=y
CONFIG_PM_DEVICE=y
CONFIG_SERIAL=n          # disable UART to allow deeper sleep
CONFIG_LOG=n             # logging keeps UART active
```

**Note:** Disabling UART removes the console. Use RTT (Real-Time Transfer) for debug output in low-power builds:

```
CONFIG_USE_SEGGER_RTT=y
CONFIG_RTT_CONSOLE=y
```

## How automatic sleep works

When all Zephyr threads call `k_msleep()` or are blocked on semaphores, the kernel calculates how long until the next wake event, then calls `pm_state_force()` or lets the idle thread select a sleep state. No application code change is needed — just enable `CONFIG_PM=y`.

## Entering System OFF (deepest sleep)

System OFF is a full power-down. The CPU does not resume — it resets on wake. Use this for ultra-low power when long intervals (minutes/hours) are acceptable.

```
#include <zephyr/pm/pm.h>
#include <hal/nrf_gpio.h>

void enter_system_off(void)
{
    /* Configure a GPIO pin as wake source */
    nrf_gpio_cfg_sense_input(NRF_GPIO_PIN_MAP(0, 11),
                           NRF_GPIO_PIN_PULLUP,
                           NRF_GPIO_PIN_SENSE_LOW);

    /* Enter System OFF – code here does not return */
    pm_state_force(0u, &(struct pm_state_info){PM_STATE_SOFT_OFF,
0, 0});
    k_sleep(K_FOREVER);
}
```

## Wake-on-motion pattern (with KXTJ3 INT pin)

1. Read + process accelerometer data
2. Configure KXTJ3 wake-on-motion interrupt
3. Enter System OFF sleep
4. KXTJ3 INT pin goes high → nRF52840 resets
5. `main()` runs again → read + process → sleep

## Measuring power savings

Use a Nordic PPK2 (Power Profiler Kit 2) or a  $\mu$ Current meter in series with the VDD supply. Compare: - No sleep: ~4 mA continuous - `CONFIG_PM=y` with 1 s sleep: ~8  $\mu$ A average - System OFF with GPIO wake: ~0.5  $\mu$ A average

## Keeping peripherals active during sleep

Some drivers support `PM_DEVICE` — they can be suspended/resumed automatically. Enable with:

```
CONFIG_PM_DEVICE=y  
CONFIG_PM_DEVICE_RUNTIME=y
```

## Why this matters

Battery life is a critical spec for wearables and IoT sensors. A coin cell (CR2032, 220 mAh) lasts hours at 4 mA but over a year at 8  $\mu$ A average.

## Practice tasks

1. Enable `CONFIG_PM=y` and measure current during `k_msleep(5000)`.
2. Compare against the same build without `CONFIG_PM`.
3. Implement System OFF with button (sw0) as wake source — confirm it wakes and boot-counts (NVS from Day 22) increments on each wake.

## Example folder

View the complete source code on GitHub: [src/2026-06-08 Power Management Sleep](#)

## Next topic

You now have all the core Zephyr building blocks. Advanced topics: MCUboot OTA updates, BLE GATT services, LwM2M cloud integration.

---